PRODUCT ROLE & ACCESS CONTROL GUIDE

Smart Restaurant Management SaaS — detailed role specification

Purpose
Explain who can access each part of the product, what each role is responsible for, how authentication is separated, and how authorization should be enforced.

Identity realm	Scope	Roles
Tenant / Restaurant	One restaurant tenant, subject to membership and branch scope	owner, manager, waiter, kitchen, cashier
Platform / SaaS	Whole SaaS platform	superadmin, ops, billing, support

Core rule
Two independent identity realms with separate identities, authentication, credentials, tokens, permissions, and administrative responsibilities. A tenant owner has no platform permissions; a platform administrator has no automatic tenant permissions.

Contents

Section	Topic
1	Role architecture
2	Tenant roles — restaurant operations
3	Platform roles — SaaS administration
4	Authentication and login flows
5	Authorization and permission design
6	Tenant and branch isolation
7	Sensitive actions and audit
8	Role lifecycle and administration
9	Implementation phases
10	Open decisions and acceptance checklist

Source and interpretation

The role names and broad responsibilities in this guide follow the product role overview supplied in the conversation. Where that overview does not specify exact action-level permissions, this document labels the proposed boundaries as recommendations. This is a product specification, not proof that every capability is already implemented; verify the repository before marking features complete.

1. Product Role Architecture

The system has two identity realms. They serve different users and protect different resources. They should not share role semantics or authentication credentials.

Dimension	Tenant realm	Platform realm
Purpose	Run restaurant operations	Operate and administer the SaaS business
Identity	Tenant user / membership	Separate PlatformAdmin identity
Scope	Tenant and assigned branches	Platform-wide
Roles	owner, manager, waiter, kitchen, cashier	superadmin, ops, billing, support
Authentication	Tenant login/session	Separate platform login/session with MFA
Authorization	Role + membership + branch scope	Platform role + permission + admin status
UI	Restaurant web/mobile apps	Platform Admin Console

1.1 Separation rules

• A restaurant owner is not a platform administrator because they own a tenant.

• A platform administrator is not automatically a member of any restaurant.

• Never accept a tenant JWT as platform authentication or a platform JWT as tenant authentication.

• Never infer platform access from tenant roles, email addresses, or frontend route visibility.

• If one person holds both responsibilities, maintain separate identity records and explicit login contexts under PA-ADR-001.

1.2 Role count

There are nine role definitions: five tenant roles and four platform roles. This does not require nine applications. Build one restaurant application and one platform console, then gate features and API actions by permissions.

2. Tenant Roles — Restaurant Operations

Tenant roles control staff actions inside their own restaurant. Effective access should be the intersection of the user's role permissions, active tenant membership, assigned branch scope, and resource state.

2.1 Owner (`owner`)

Purpose: Primary business authority for the restaurant tenant.

• Broad control over orders, kitchen, bills, customer payments, staff, settings, reports, subscriptions, branches, inventory, employees, shifts, loyalty, and reviews.

• Can configure the restaurant and delegate operational responsibilities.

• **Recommended safeguards:** ownership transfer, account recovery, payout/bank changes, and tenant deletion should require explicit confirmation and strong authentication.

2.2 Manager (`manager`)

Supplied definition: Same permissions as owner. This creates a role distinction without a permission distinction.

• Temporary parity is possible, but is not ideal for production.

• **Recommended boundary:** daily operations, menu, orders, staff coordination, inventory, reports, and assigned branch operations. Reserve ownership transfer, subscription ownership, security administration, sensitive payment configuration, and tenant deletion for the owner or explicitly authorized users.

2.3 Waiter (`waiter`)

Purpose: Customer service and order handling.

• Read/create orders, serve customers, and access kitchen/order information needed for service.

• The supplied overview also lists branches, inventory, employees, shifts, loyalty, and reviews; these broad labels need action-level clarification.

• **Recommended restrictions:** no subscription management, staff role assignment, sensitive reports, tenant settings, or refund/payment administration unless explicitly granted.

2.4 Kitchen (`kitchen`)

Purpose: Prepare confirmed orders and update kitchen workflow state.

• Read relevant orders and kitchen queue information; update preparation status such as preparing/ready.

• Read inventory information needed for preparation, if enabled.

• **Recommended restrictions:** no customer payment details, subscription data, staff administration, or broad customer personal information.

2.5 Cashier (`cashier`)

Purpose: Issue bills and record customer payments.

• Read orders, issue bills, record payments, read reports, and redeem loyalty benefits if permitted.

• The supplied overview also lists kitchen, branches, loyalty, and subscription read access; expose only the fields needed for cashier workflows.

• **Recommended restrictions:** no platform subscription mutation, platform refunds, or ownership/security changes. Restaurant payment reversals should be separately permissioned and audited.

2.6 Tenant role responsibility matrix

Capability	Owner	Manager*	Waiter	Kitchen	Cashier
Orders	Full	Operational	Read/create/serve	Read relevant	Read
Kitchen workflow	Full	Operational	Read needed	Read/update	Read needed
Bills/customer payments	Full	Assigned	Limited service context	No	Issue/record
Menu/settings	Full	Operational*	No/limited	No	No
Staff/shifts	Full	Operational*	Assigned visibility	No/limited	No/limited
Inventory	Full	Operational*	Read/limited*	Read	Read/limited*
Reports	Full	Operational*	No/limited	No	Read
Subscriptions	Manage	Read/limited*	No	No	Read (supplied)
Branches	Manage	Assigned*	Assigned visibility	Assigned visibility	Read/assigned
Loyalty/reviews	Full	Operational*	Assigned actions*	No	Read/redeem

*Exact actions are not specified in the supplied role summary. These are recommended boundaries and must be approved and translated into explicit permissions.

2.7 Multi-branch access

A tenant may have one or multiple branches. A role should not grant all-branch access by default. Validate branch assignment for each request. Owner access may be tenant-wide; other roles can be limited to assigned branches.

2.8 Two different payment domains

Restaurant customer payments pay for meals/orders and are handled by restaurant staff, typically the cashier. SaaS subscription payments are paid by the restaurant to the SaaS provider and are verified by platform billing staff. Keep their records, permissions, reports, and audit trails separate.

3. Platform Roles — SaaS Administration

Platform roles control the Admin Console and platform-wide resources. They must be stored and authorized independently from tenant roles.

3.1 Superadmin (`superadmin`)

Purpose: Highest-trust platform administrator.

• Full access to dashboard, tenants, plans, subscriptions, invoices, payments, refunds, platform users, audit, settings, diagnostics, and analytics.

• Can create, suspend, and reactivate tenants and manage platform functions.

• Should manage privileged access and security-sensitive settings only with MFA, step-up authentication where appropriate, and audit records.

3.2 Operations (`ops`)

Purpose: Tenant lifecycle and operational administration.

• Supplied capabilities include tenant suspend/reactivate, force logout, export, create/reactivate, diagnostics, and viewing plans, subscriptions, invoices, audit, and settings.

• **Recommended restriction:** no payment confirmation, refund approval/completion, or unrestricted payment instrument access unless separately granted.

3.3 Billing (`billing`)

Purpose: SaaS subscription and finance operations.

• View/search tenants; view plans; manage subscriptions and invoices; confirm payments; request, approve, and complete refunds according to workflow permissions.

• Has dashboard, audit, settings, and diagnostics access as specified in the supplied role overview.

• **Recommended control:** separate refund requester, approver, and completer for high-risk cases; prohibit self-approval.

3.4 Support (`support`)

Purpose: Customer support and limited investigation.

• View dashboard, tenant information/search, plans, subscriptions, invoices, and audit records.

• Use diagnostics only within an approved support scope.

• **Recommended restriction:** read-only by default; no tenant suspension, payment confirmation, refund, plan mutation, or unrestricted tenant export.

3.5 Platform permission summary

Capability	Superadmin	Ops	Billing	Support
Dashboard	Full	Allowed	Allowed	Allowed
Tenant view/search	Full	Operational	View/search	View/search
Tenant create/suspend/reactivate	Full	Allowed per policy	No	No
Tenant force logout/export	Full	Allowed per policy	No	No
Plans	Manage	View	View	View
Subscriptions	Manage	View	Manage	View
Invoices	Manage	View	Manage	View
Payment confirmation	Full	No by default	Allowed	No
Refund request/approve/complete	Full	No by default	Workflow permissions	No
Platform users	Manage	No by default	No by default	No
Audit	Full	Scoped	Scoped	Scoped
Settings/diagnostics	Full	Scoped	Scoped	Read-only scoped
Analytics	Full	Operational	Financial	Limited

This matrix combines the supplied selected-permission lists with recommended least-privilege boundaries. The backend permission map (proposed as `PLATFORM_ROLE_PERMISSIONS`) and its tests must be authoritative.

3.6 One console, permission-gated

• Use one Admin Console shell and shared components; do not build four separate admin apps.

• Show navigation according to permissions, but enforce every action on the backend.

• A hidden frontend page is not authorization. Direct API requests must still be denied.

• Prefer action permissions such as `payment.confirm`, `refund.request`, `refund.approve`, and `refund.complete` over broad page-only permissions.

4. Authentication and Login Flows

The two realms use independent login flows. The following routes and cookie names are from the supplied project description and should be verified against the current implementation.

4.1 Tenant login

Step	Expected behavior
1. Login	Tenant user submits credentials to the tenant authentication endpoint.
2. Identity	Backend resolves user and relevant tenant membership.
3. Context	Load tenant role, membership status, and branch assignments.
4. Session	Issue tenant-scoped credentials with necessary claims.
5. Request	Validate identity, membership, tenant/branch scope, permission, and resource state.

4.2 Platform login

Step	Expected behavior
1. Login	POST `/api/platform/auth/login` with platform credentials.
2. Identity	Look up the separate PlatformAdmin identity.
3. MFA	Verify the six-digit TOTP authenticator code.
4. Status/role	Check active status and platform role. Current PA-0.12 description allows only `superadmin`.
5. Session	Set platform-specific access and refresh cookies; never reuse tenant credentials.

Step	Expected behavior
6. Authorization	Check platform permission for every protected API action.

4.3 Admin route

The described development route is `/admin`, for example `http://{ip}:5173/admin`. A URL path is not a security boundary. The API must authenticate and authorize every platform request. Production domain/origin decisions remain subject to the platform-domain ADR.

4.4 Bootstrap

• Development may use a controlled local bootstrap process.

• Production bootstrap must be explicitly enabled, one-time, secret-managed, audited, and disabled after successful creation.

• Never ship a known default production password or expose bootstrap credentials through a public endpoint.

• Require MFA enrollment and document emergency recovery.

5. Authorization and Permission Design

A role is a named bundle of permissions. The backend must evaluate authorization for every protected operation.

Check	Tenant request	Platform request
Identity	Valid tenant user/session	Valid PlatformAdmin/session
Status	User and membership active	Platform admin active
Realm	Tenant realm only	Platform realm only
Scope	Correct tenant and assigned branch/resource	Platform-wide or explicitly scoped admin capability
Permission	Tenant role/action permission	Platform role/action permission
Resource state	Order/bill/menu state permits action	Tenant/subscription/invoice/payment state permits action

5.1 Suggested permission identifiers

• Tenant: `orders.read`, `orders.create`, `orders.cancel`, `kitchen.read`, `kitchen.update`, `bills.issue`, `payments.record`, `staff.manage`, `branches.manage`, `subscriptions.read`.

• Platform: `tenants.view`, `tenants.create`, `tenants.suspend`, `tenants.reactivate`, `tenants.export`, `subscriptions.manage`, `invoices.manage`, `payments.confirm`, `refund.request`, `refund.approve`, `refund.complete`, `platform_users.manage`, `audit.read`, `settings.manage`, `diagnostics.read`.

These identifiers are proposals, not a claim that they already exist in the repository.

5.2 Deny by default

• Unknown roles or permissions are denied.

• Inactive users, memberships, or platform admins are denied.

• A tenant ID supplied by the browser is a selector, not authorization proof.

• A platform role is never inferred from a tenant role.

• Sensitive actions may require fresh authentication or step-up verification.

5.3 Centralize policy

Define permissions in a centralized backend policy module. Frontend route guards and navigation can mirror that policy, but the backend remains authoritative. Add tests for allowed and denied actions for every role.

6. Tenant and Branch Isolation

• Every tenant-owned record must be associated with a tenant and queried through authorized tenant context.

• Resolve tenant context from authenticated membership; never trust a client-supplied tenant ID alone.

• For multi-branch tenants, validate branch assignment for branch-scoped roles and records.

• Use tenant-aware cache keys, background jobs, notifications, exports, object-storage paths, and audit metadata.

• Use tenant-aware foreign keys and constraints where practical to prevent cross-tenant references.

• Consider PostgreSQL RLS as defense in depth, with transaction-local tenant context and safe connection-pool handling.

• Workers must establish and validate tenant context before processing tenant-owned work.

6.1 Example

A waiter assigned to Branch A must not retrieve Branch B orders by changing an order or branch ID in a request. The backend verifies tenant membership, branch assignment, and the required order permission for the target resource.

6.2 Platform access is not tenant membership

Platform support or operations access does not automatically make an administrator a tenant user. If support tooling later needs tenant data, implement explicit, scoped, audited access. The current direction defers impersonation and favors read-only diagnostics.

7. Sensitive Actions and Audit Requirements

Action	Recommended control
Suspend/reactivate tenant	Dedicated permission, reason, actor/time, and tenant notification.
Confirm SaaS payment	Billing permission, amount/reference validation, duplicate prevention, audit.
Refund	Separate request/approve/complete actions; prohibit self-approval; threshold policy remains open.
Change plan/subscription	Record before/after values, effective date, actor, reason, entitlement effects.
Export tenant data	Explicit permission, scope confirmation, secure delivery, expiry, audit.
Force logout/revoke sessions	Dedicated permission, reason, affected principal, audit.
Change platform admin role/status	Privileged permission, step-up, audit, and notification.
Change payment QR/bank details	Strong verification and audit; separate SaaS collection details from restaurant payment settings.
Archive/delete tenant	Check export, retention, and legal holds; confirm and audit.

7.1 Audit event minimum

• Actor identity and realm (tenant user or platform admin).

• Action and target resource identifiers; tenant/branch context where relevant.

• Timestamp, outcome, and reason for sensitive actions.

• Request/correlation ID and carefully selected security metadata.

• Before/after values for business changes, excluding secrets and credentials.

7.2 Transaction behavior

For business mutations, write the audit record in the same database transaction where practical, so the mutation and audit entry succeed or fail together. Security events that must survive a failed business transaction may need a separate transaction or security-event pipeline, consistent with the audit ADR.

8. Role Lifecycle and Administration

8.1 Tenant user lifecycle

• Create/invite a user within a tenant and assign a membership with a tenant role.

• Manage membership status and branch assignments separately from global user identity.

• Role changes must be permission-checked and audited.

• Removing a membership should revoke access to that tenant without necessarily deleting the global identity or other memberships.

• A user in multiple tenants can have a different role and branch scope in each membership.

8.2 Platform admin lifecycle

• Provision platform admins through a controlled process, not public restaurant registration.

• Each platform admin has a separate platform role and active/suspended status.

• Role changes, deactivation, MFA reset, and session revocation are privileged and audited.

• Identify at least two independent recovery paths before commercial launch.

8.3 Fixed platform roles

PA-ADR-006 proposes fixed platform role permissions in code through `PLATFORM_ROLE_PERMISSIONS` for the initial phase. A database-backed role editor is deferred. This is reasonable for four fixed roles; changes should be code-reviewed, tested, deployed, and audited.

8.4 Roles versus subscription entitlements

A role answers what a person may do. A subscription entitlement answers what the tenant's plan includes. A user may have permission to use inventory, but the tenant plan may not include inventory. The request must pass both checks.

9. Implementation Guidance and Phase Boundaries

Phase	Role-related scope
PA-0 — Foundation	Separate platform identity; separate tenant/platform auth; platform MFA; permission catalogue; membership/branch checks; audit foundation; secure bootstrap.
PA-1 — Commercial MVP	Superadmin console and core SaaS operations. If other platform roles are not enabled, restrict console/API access to superadmin.
PA-2 — Operational maturity	Enable ops, billing, and support access; permission-gated console navigation and APIs; support workflows and access reviews.
PA-3 — Advanced	Consider DB-backed role editor, enterprise SSO, advanced scoped support access, and further governance when justified.

9.1 Avoid overbuilding

• One tenant application can serve five tenant roles using permission-aware navigation and backend authorization.

• One platform console can serve four platform roles using permission-aware navigation and backend authorization.

• Do not enable a role until its permission matrix, API enforcement, UI behavior, audit behavior, and tests are complete.

9.2 Required authorization tests

• Tenant owner cannot call platform endpoints or authenticate with platform credentials.

• Platform admin cannot act as a tenant user without explicit tenant membership.

• Waiter cannot read another tenant's or unauthorized branch's orders.

• Kitchen cannot issue bills or record payments.

• Cashier cannot change tenant ownership or platform subscriptions.

• Support cannot suspend tenants or confirm payments.

• Ops cannot approve/complete refunds unless explicitly granted.

• Billing cannot change platform admin roles unless explicitly granted.

• Inactive principals are denied; missing subscription entitlements block features despite user permission.

10. Open Decisions and Acceptance Checklist

10.1 Decisions to finalize

• Decide whether manager remains equivalent to owner or receives a narrower permission set.

• Define exact waiter, kitchen, and cashier action-level permissions.

• Define branch assignment behavior for every tenant role.

• Confirm whether ops, billing, and support are enabled in PA-1 or deferred to PA-2; recommendation: defer full console access to PA-2.

• Define refund threshold and separation of duties before dual approval is enabled.

• Define tenant export scope, authorization, secure delivery, and expiry.

• Clarify settings and diagnostics permissions for ops, billing, and support.

• Finalize production bootstrap, MFA recovery, secondary Super Admin, and break-glass ownership.

10.2 Definition of done

• Two independent identity realms with separate authentication and credentials.

• Every protected endpoint enforces realm, status, permission, tenant, and branch scope as applicable.

• Backend permission catalogue is centralized and version-controlled.

• Frontend visibility mirrors policy but is not relied on for security.

• Cross-tenant and cross-branch negative tests pass.

• Privileged mutations create auditable events.

• Role changes and session revocations take effect predictably.

• Subscription entitlements are checked separately from user permissions.

• Documentation distinguishes shipped functionality from planned capabilities.

Final architecture statement

The Smart Restaurant Management SaaS has two independent identity realms: tenant identities for restaurant operations and platform identities for SaaS administration. Each realm has its own roles and permissions. The backend enforces authorization on every protected action, with tenant and branch isolation, subscription entitlement checks, and audit controls where relevant.

Document status: Product role specification and implementation guidance. Recommended boundaries require product approval and code-level verification.